โครงงาน

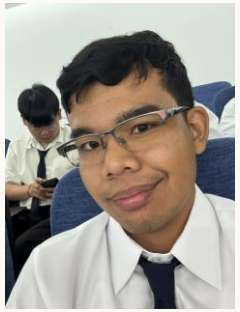
PDF NOTE STUDIO

วิชา พื้นฐานการออกแบบและพัฒนาซอฟต์แวร์

03376116

สมาชิกในกลุ่ม

นายปัญญา สุขถาวร 69030358



นายศุภกรณ์ โพธิ์เจริญ 69030413



นายศักย์ศรณ์ ทองคำพูล 69030229



นายอิทธิเดช อิ่มสบาย 69030431



ความเป็นมา

ปัจจุบันนักศึกษาใช้เอกสาร PDF เป็นสื่อประกอบการเรียนและทบทวนบทเรียน เมื่อมีเอกสารหลายไฟล์ การจดบันทึกแยกจากเอกสารทำให้ค้นหาคำอธิบายและกลับไปยังหน้าต้นฉบับได้ยาก อีกทั้งการรวบรวมเนื้อหาสำหรับเตรียมสอบต้องใช้เวลา คณะผู้จัดทำจึงพัฒนา PDF Note Studio เพื่อรวมการอ่าน PDF การจดโน้ตรายหน้า และการค้นหาข้อมูลไว้ในโปรแกรมเดียว พร้อมนำ AI มาช่วยสรุปเนื้อหาและเพิ่มเครื่องมือทบทวนบทเรียน โดยใช้ PostgreSQL จัดเก็บโน้ตและข้อมูลที่เกี่ยวข้องให้สามารถกลับมาใช้งานได้ภายหลัง

วัตถุประสงค์

- เพื่อพัฒนาโปรแกรมอ่านเอกสาร PDF ที่สามารถจดและบันทึกโน้ตแยกตามหน้าได้
- เพื่อให้ผู้ใช้ค้นหาข้อความใน PDF รวมถึงโน้ตและผลสรุปที่บันทึกไว้จากหลายเอกสารได้
- เพื่อประยุกต์ใช้ AI ช่วยสรุปเนื้อหาและตอบคำถามจากเอกสารพร้อมอ้างอิงเลขหน้า
- เพื่อสร้างเครื่องมือช่วยทบทวนบทเรียน ได้แก่การปักธง แฟลชการ์ด และการรวมชีทก่อนสอบ
- เพื่อประยุกต์ใช้ PostgreSQL จัดเก็บและเรียกใช้ข้อมูลของโปรแกรมอย่างเป็นระบบ

ฟังก์ชันของระบบและเงื่อนไขการใช้งาน

1. การค้นหาคำ
2. สรุปรายหน้าที่พบ
3. ไฮไลค์คำที่พบ
4. ปุ่มล้างการค้นหา
5. วิเคราะห์หน้าด้วย ai
6. ปุ่มยกเลิกการวิเคราะห์ ai
7. พื้นที่จดโน้ต
8. เปิดไฟล์ PDF
9. แถบควบคุมหน้า PDF
10. ระบบย่อ-ขยายหน้าเอกสาร

เงื่อนไขการใช้งาน

ฟังก์ชันค้นข้อความและ AI ใช้ข้อความที่ดึงจาก PDF รุ่นปัจจุบันยังต้องเพิ่ม OCR เพื่อรองรับเอกสารสแกนที่ไม่มีชั้นข้อความ การสรุปและถามชีทต้องมี Ollama และโมเดลพร้อมใช้งาน และการจัดเก็บข้อมูลต้องเชื่อมต่อ PostgreSQL ได้

ขีดความสามารถของโครงงาน

ฟังก์ชัน	ความสามารถ
อ่านเอกสาร PDF	เปิดไฟล์แสดงภาพย่อ เลือกหน้า ซูม และแสดงเต็มจอ
จดโน้ตรายหน้า	บันทึกโน้ตลง PostgreSQL และเรียกดูเมื่อเปิดเอกสารเดิม
ค้นหาใน PDF	ค้นคำ แสดงหน้าที่พบ และไฮไลท์ตำแหน่งข้อความ
ค้นหาข้ามเอกสาร	ค้นโน้ตและผล AI ที่บันทึกไว้พร้อมเปิดกลับไปยังหน้าต้นฉบับ
สรุปด้วย AI	ใช้ Ollama สรุปข้อความของหน้าปัจจุบันเป็นภาษาไทย
ปักธงข้อสงสัย	บันทึกคำถามประจำหน้า และเปลี่ยนสถานะเป็นเข้าใจแล้ว
แฟลชการ์ด	สร้างบัตรคำถาม คำตอบจากโน้ต และบันทึกสถานะการทบทวน
รวมชีทก่อนสอบ	เลือกโน้ตและผลสรุปจากหลายเอกสาร แล้วส่งออกเป็นไฟล์ .txt
ถามชีทพร้อมเลขหน้า	ให้ AI ตอบจากข้อความในเอกสาร พร้อมตรวจข้อความอ้างอิงและเปิดหน้าที่เกี่ยวข้อง
เก็บผล AI ที่เราบันทึก	เก็บไว้บนเครื่องเพื่อลองบันทึกใหม่หรือส่งออกเป็นไฟล์ JSON

โมเดลพัฒนาซอฟต์แวร์

Waterfall Model

นายปัญญา สุขถาวร 69030358

- เขียนโค้ดจัดทำ Desktop app

นายศุภกรณ์ โพธิ์เจริญ 69030413

- จัดทำฐานข้อมูล Database

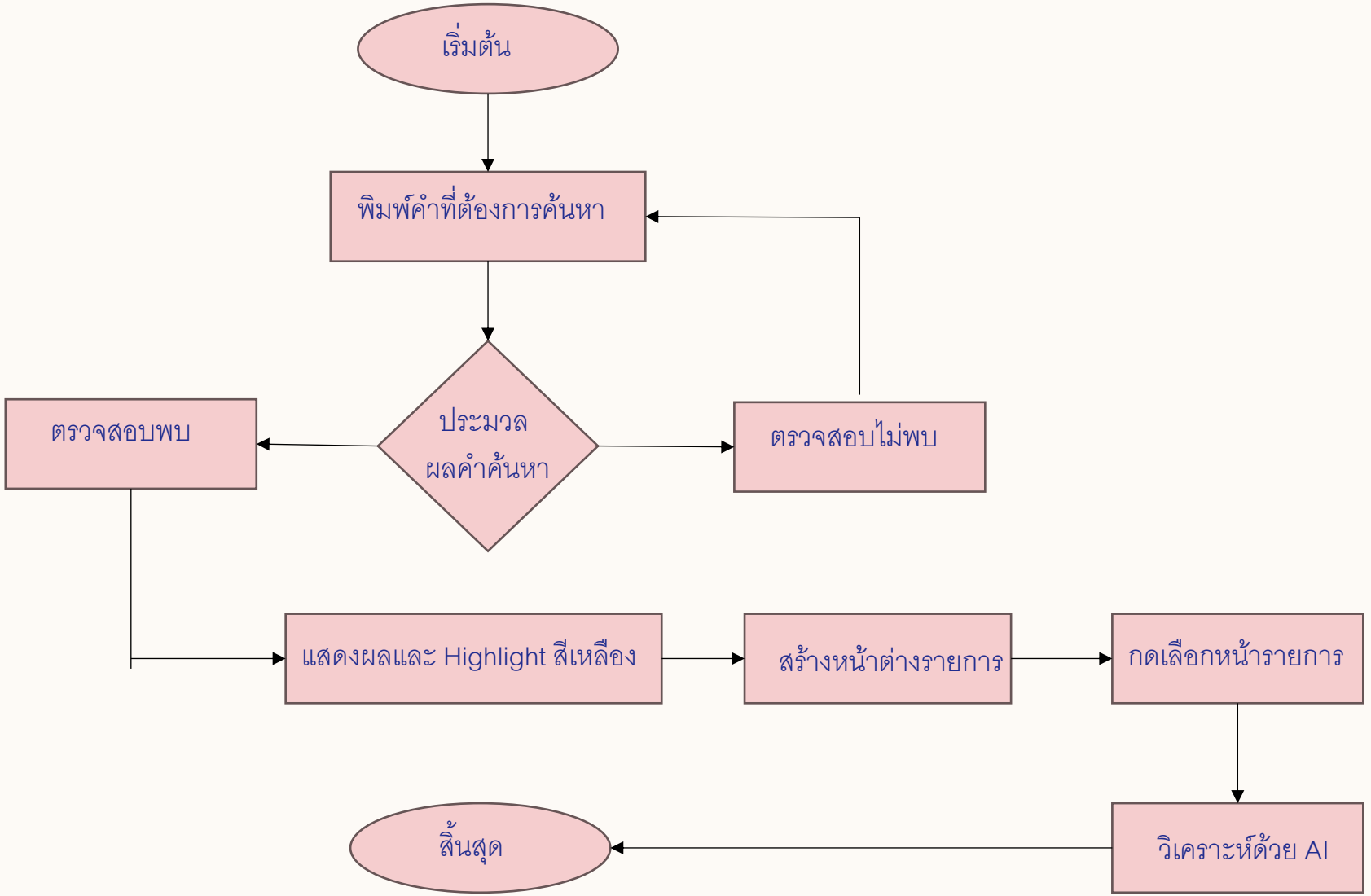
นายศักย์ศรณ์ ทองคำพูล 69030229

- จัดทำการเพิ่มฟังก์ชัน

นายอิทธิเดช อิ่มสบาย 69030431

- จัดทำสไลด์นำเสนอ

โครงสร้างโครงงานในภาพรวม



การติดตั้งโปรแกรม

ขั้นตอนที่ 1 . ติดตั้ง Python version 3.10 ขึ้นไป

ขั้นตอนที่ 2. อัพเดต pip ในเครื่องด้วยคำสั่ง python -m pip install --upgrade pip

ขั้นตอนที่ 3. ติดตั้งแพ็คเกจที่ต้องใช้งาน pip install PyQt6 pymupdf ollama

ขั้นตอนที่ 4. ติดตั้งและเปิดใช้งาน Ollama

4.1 ดาวน์โหลดโปรแกรม Ollama จากเว็บไซต์: https://ollama.com/download

4.2 ติดตั้งโปรแกรม Ollama ลงในเครื่อง

4.3 เปิด Terminal หรือ Command Prompt แล้วสั่งดาวน์โหลดโมเดลภาษาไทย (qwen2.5:7b):

4.4 ใช้คำสั่ง ollama run qwen2.5:7b

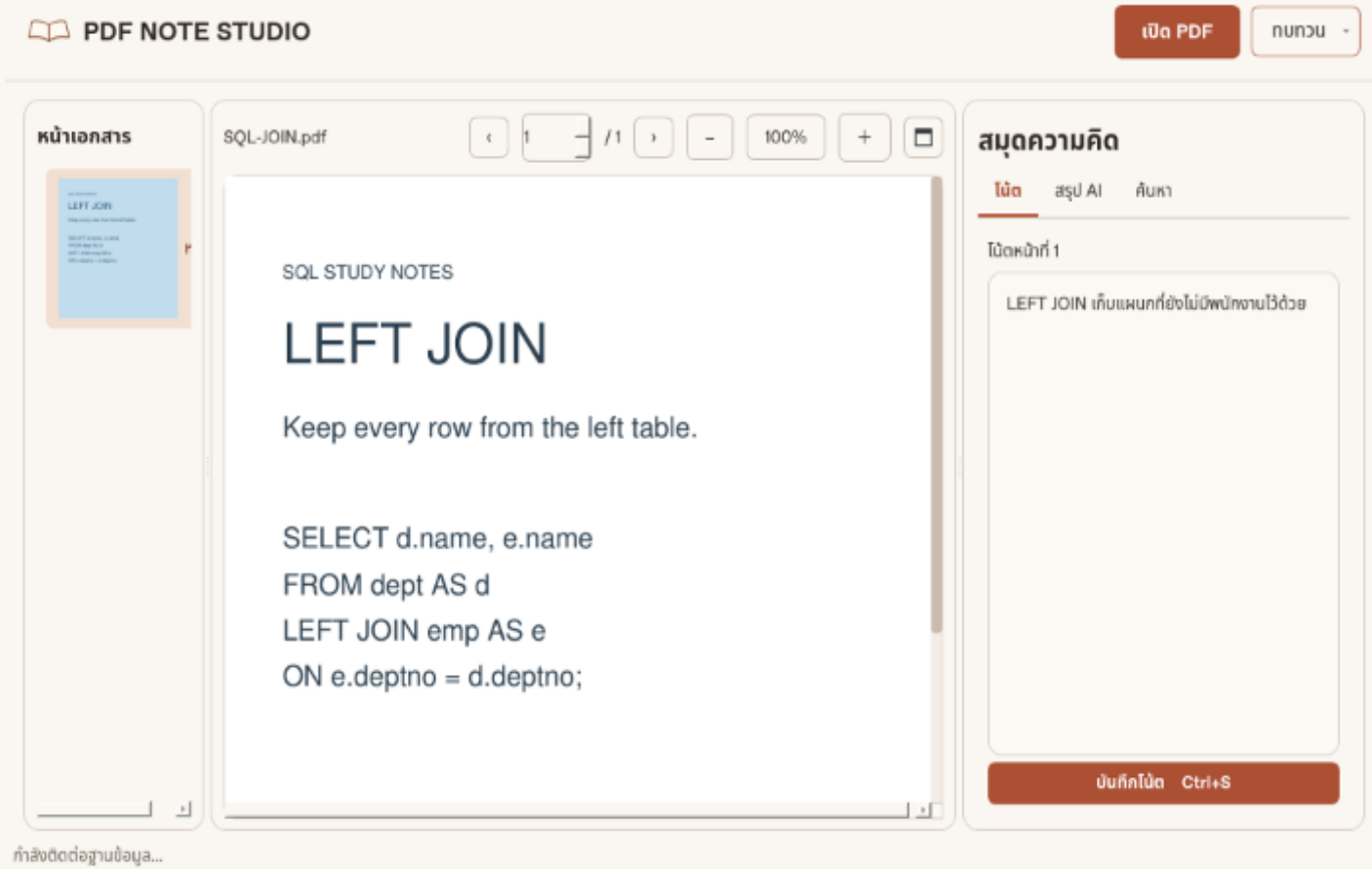
ขั้นตอนที่ 5. ดาวน์โหลดโค้ดและรันโปรแกรม

5.1 สร้างไฟล์โค้ดชื่อ app.py หรือ project1.py ในโฟลเดอร์ที่คุณต้องการ แล้วนำโค้ดทั้งหมดไปวางและบันทึก

5.2 เปิด Terminal / Command Prompt แล้วเข้าไปยังโฟลเดอร์ที่เก็บไฟล์ เช่น: cd /path/to/your/folder

5.3 สั่งรันโปรแกรม: macOS / Linux: bash python3 app.py Windows: bash python app.py

สาธิตการอ่านและจดโน้ตในหน้าเดียว



1. เปิด PDF

เลือก SQL-JOIN.pdf

2. อ่านหน้า LEFT JOIN

ชี้หน้าเอกสารที่แสดงตรงกลาง

3. จดและบันทึก

พิมพ์โน้ตด้านขวา แล้วกด Ctrl+S

จากโน้ตหนึ่งหน้าไปถึงชุดทบทวน

ค้นหาทุกชีท	เลือกค้นโน้ตและผล AI ทุกชีท พิมพ์ JOIN แล้วกด Enter คลิกผลลัพธ์เพื่อกลับไปหน้าที่จดไว้
ทบทวนด้วยตนเอง	ปักธงหน้าที่ยังสงสัย หรือเปลี่ยนโน้ตเป็นแฟลชการ์ด เลือกโน้ตและผลสรุปหลายเล่มเพื่อส่งออกชีท .txt
ใช้ AI ช่วยอ่าน	สรุปหน้าปัจจุบัน หรือถามชีทแล้วตรวจข้อความอ้างอิง ต้องเปิด Ollama และมีโมเดล qwen2.5:7b

เมื่อกดบันทึก โน้ตเดินทางไปไหน?

หน้าต่าง PyQt6	studio_ui.py รับการพิมพ์และการกด Ctrl+S project1.py จับคู่ข้อความกับเอกสารและหน้าปัจจุบัน
งานเบื้องหลัง	background.py ทำงานฐานข้อมูลแยกจากหน้าต่าง database/store.py ส่งคำสั่ง SQL ไปยัง PostgreSQL
การเปิดอ่านอีกครั้ง	ใช้รหัสเอกสารและเลขหน้าดึงโน้ตมาแสดง PyMuPDF วาดหน้า PDF ส่วน Ollama รับงานสรุปข้อความ

แสดงเอกสารในช่องอ่านตรงกลาง

project1.py · render_page()

```
page = self.doc.load_page(self.current_page)
pix = page.get_pixmap(dpi=zoom_dpi, colorspace=fitz.csRGB, alpha=False)
# ... แปลงภาพจาก PyMuPDF เป็น QPixmap ...
self.lbl_pdf_page.setPixmap(pixmap)
self.reader_stack.setCurrentWidget(self.scroll_area)
```

ส่วนที่ตัดออกคือการแปลงรูปภาพและการวาดไฮไลต์คำค้น

บันทึกซ้ำหน้าเดิม แล้วโน้ตถูกแก้ที่แถวเดิม

database/store.py · save_note()

```
INSERT INTO notes(document_id, page_number, content) VALUES (%s, %s, %s)
ON CONFLICT (document_id, page_number) DO UPDATE
SET content=EXCLUDED.content, updated_at=CURRENT_TIMESTAMP
```

ครั้งแรก → เพิ่มโน้ตของหน้านั้น

ครั้งถัดไป → อัปเดตเนื้อหาและเวลาที่แก้ล่าสุด

เปลี่ยนโน้ตเป็นคำตอบของแฟลชการ์ด

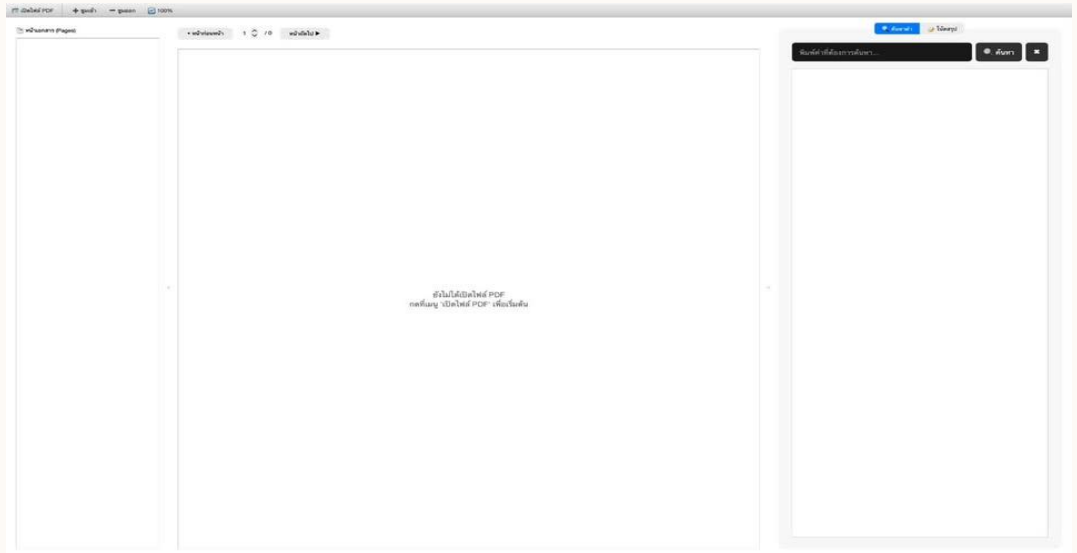
study_tools.py · make_card()

```
answer = self.txt_notes.toPlainText().strip()
doc, page = self.document_id, self.current_page+1
# ... รับคำถามจากหน้าต่าง แล้วบันทึกบัตรด้วย ...
self.database.create_card(doc, page, question.strip(), answer)
```

ตัวอย่างคำถาม LEFT JOIN เก็บแถวใดไว้เสมอ?

คำตอบจากโน้ต ทุกแถวของตารางด้านซ้าย แม้ไม่มีแถวที่จับคู่ได้

โค้ด



- หน้าแอพตอนเปิดใช้งาน

```python
# ฟังก์ชัน: ระบบตามสรุป Ollama AI
# ──────────────────────────────────────────────────────────
def analyze_page_with_ollama(self):
    """ดึงข้อความจากหน้าปัจจุบัน แล้วสั่งให้ Ollama AI สรุปเนื้อหาในเบื้องหลัง"""
    if not self.doc:
        QMessageBox.warning(self, "แจ้งเตือน", "กรุณาเปิดไฟล์ PDF ก่อนใช้งาน AI")
        return

    page = self.doc.load_page(self.current_page)
    page_text = page.get_text().strip()

    # ตรวจก่อนว่าในหน้านี้มีข้อความให้ดำเนินหรือไม่
    if not page_text:
        QMessageBox.warning(
            self, "แจ้งเตือน",
            "ไม่พบข้อความในหน้านี้ (อาจเป็นรูปภาพล้วน หรือเอกสารสแกน)"
        )
        return

    # ปรับสถานะปุ่ม: ปิดปุ่มวิเคราะห์ทั้งหมด และเปิดใช้งานปุ่ม 'ยกเลิก'
    self.btn_ai.setEnabled(False)
    self.btn_ai.setText("⏳ กำลังวิเคราะห์...")
    self.btn_cancel_ai.setEnabled(True)

    self.txt_notes.append(f"\n─── 🤖 [AI สรุปเนื้อหา หน้า {self.current_page + 1}] ───")
    self.txt_notes.append("กำลังประมวลผล กรุณารอสักครู่ (กด 'ยกเลิก' ได้)...\n")

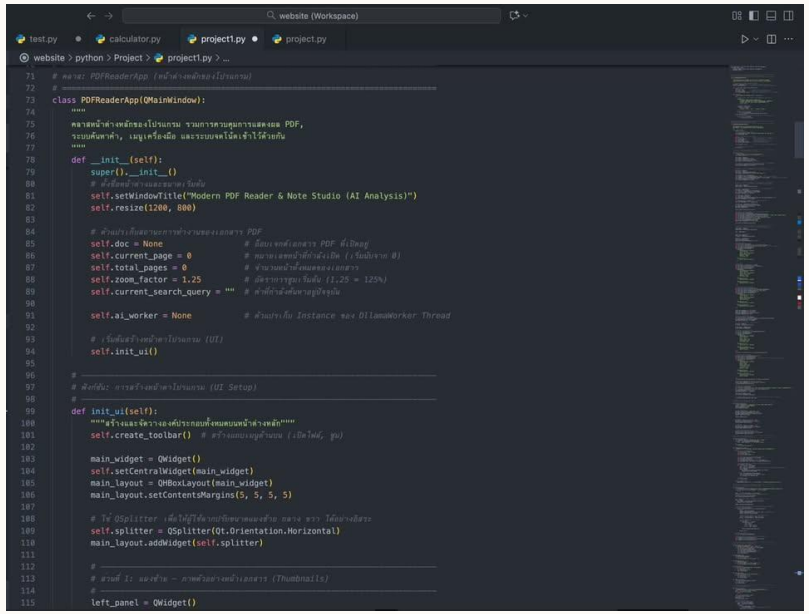
    # สร้างและเริ่มต้นการทำงานของเธรด OllamaWorker
    self.ai_worker = OllamaWorker(text=page_text, model="qwen2.5:7b")
    self.ai_worker.finished.connect(self.on_ai_success)
    self.ai_worker.error.connect(self.on_ai_failed)
    self.ai_worker.start()

def cancel_ai_processing(self):
    """ฟังก์ชันสำหรับยกเลิกการทำงานของ AI กลางคันอย่างปลอดภัย"""
    if self.ai_worker and self.ai_worker.isRunning():
        self.ai_worker.is_cancelled = True
        self.ai_worker.terminate()  # สั่งหยุดเธรดทันที
        self.ai_worker.wait(400)    # รอให้เธรดปิดตัวเอง

        # ลบข้อความที่ค้างว่า 'กำลังประมวลผล' ออก แล้วใส่ข้อความยกเลิกแทน
        current_text = self.txt_notes.toPlainText()
        cleaned_text = current_text.replace(
            "กำลังประมวลผล กรุณารอสักครู่ (กด 'ยกเลิก' ได้)...\n", ""
        )
```

- ดึงข้อมูลปัจจุบันแล้วสั่งให้ ai ollama เป็นคนสรุปหน้า
- ใช้ภาษา python
- เชื่อม Database PostgreSQL

โค้ด



- คลาสหน้าต่างหลักของโปรแกรม รวมการควบคุมการแสดงผล PDF, ระบบค้นหาคำ, เมนูเครื่องมือ และระบบจดโน้ตเข้าไว้ด้วยกัน

```python
def create_toolbar(self):
    """สร้างปุ่มลัดด้านบน: เปิดไฟล์ PDF, ซูมเข้า, ซูมออก และรีเซ็ตขนาด"""
    toolbar = QToolBar("เครื่องมือหลัก")
    toolbar.setMovable(False)
    self.addToolBar(toolbar)

    # เมนูเปิดไฟล์ PDF
    btn_open = QAction("📂 เปิดไฟล์ PDF", self)
    btn_open.setShortcut(QKeySequence("Ctrl+O"))
    btn_open.triggered.connect(self.open_pdf_dialog)
    toolbar.addAction(btn_open)

    toolbar.addSeparator()

    # เมนูซูมเข้า
    btn_zoom_in = QAction("🔍 ซูมเข้า", self)
    btn_zoom_in.triggered.connect(self.zoom_in)
    toolbar.addAction(btn_zoom_in)

    # เมนูซูมออก
    btn_zoom_out = QAction("🔍 ซูมออก", self)
    btn_zoom_out.triggered.connect(self.zoom_out)
    toolbar.addAction(btn_zoom_out)

    # เมนูรีเซ็ตการซูมเป็น 100%
    btn_zoom_reset = QAction("🔄 100%", self)
    btn_zoom_reset.triggered.connect(self.zoom_reset)
    toolbar.addAction(btn_zoom_reset)
```

- สร้างปุ่มเปิดไฟล์ pdf ซูมเข้า ซูมออก และ การรีเซ็ตการซูม

โค้ด

```python
# ──────────────────────────────────────────────────────────────
def open_pdf_dialog(self):
    """เปิดหน้าต่างเลือกไฟล์เพื่อเลือกไฟล์ PDF จากเครื่อง"""
    file_path, _ = QFileDialog.getOpenFileNames(
        self, "เลือกเอกสาร PDF", "", "PDF Files (*.pdf)"
    )
    if file_path:
        self.load_pdf(file_path[0])

def load_pdf(self, path):
    """โหลดไฟล์ PDF, นับจำนวนหน้า, สร้างภาพ Thumbnail และเรนเดอร์หน้าแรก"""
    try:
        self.doc = fitz.open(path)
        self.total_pages = len(self.doc)
        self.current_page = 0
        self.current_search_query = ""

        # ปรับปรุงค่าใน SpinBox และ Label แสดงหน้าทั้งหมด
        self.spin_page.setMaximum(self.total_pages)
        self.spin_page.setValue(1)
        self.lbl_page_total.setText(f"/ {self.total_pages}")
        self.setWindowTitle(f"PDF Reader - {os.path.basename(path)}")

        # สร้างภาพย่อ Thumbnail ของทุกหน้าลงใน ListWidget ทางซ้าย
        self.thumbnail_list.clear()
        for i in range(self.total_pages):
            page = self.doc.load_page(i)
            pix = page.get_pixmap(dpi=36)  # ใช้ DPI ต่ำ (36 DPI) เพื่อให้โหลดเร็วและประหยัดแรม
            qimg = QImage(
                pix.samples, pix.width, pix.height, pix.stride,
                QImage.Format.Format_RGB888
            )
            icon = QIcon(QPixmap.fromImage(qimg))
            item = QListWidgetItem(icon, f"หน้า {i + 1}")
            self.thumbnail_list.addItem(item)

        # วาดหน้าแรกขึ้นจอ
        self.render_page()

    except Exception as e:
        QMessageBox.critical(self, "เกิดข้อผิดพลาด", f"ไม่สามารถเปิดไฟล์ได้:\n{str(e)}")
```

- คำนวณจำนวนหน้า pdf

```python
def render_page(self):
    """เรนเดอร์หน้าเอกสารปัจจุบันตามค่าการซูม (DPI) พร้อมวาดไฮไลต์สีเหลืองตรงจุดที่พบคำ"""
    if not self.doc or self.current_page >= self.total_pages:
        return

    page = self.doc.load_page(self.current_page)
    # คำนวณความละเอียด DPI ตามอัตราการซูม เพื่อให้ภาพคมชัดเสมอ
    zoom_dpi = int(100 * self.zoom_factor)
    pix = page.get_pixmap(dpi=zoom_dpi)

    # แปลงข้อมูลภาพจาก PyMuPDF เป็น QImage และ QPixmap ของ PyQt
    qimg = QImage(
        pix.samples, pix.width, pix.height, pix.stride,
        QImage.Format.Format_RGB888
    )
    pixmap = QPixmap.fromImage(qimg)

    # หากมีคำที่ค้นหาอยู่ ให้วาดแถบไฮไลต์สีเหลืองทับตำแหน่งคำนั้น
    if self.current_search_query:
        rects = page.search_for(self.current_search_query)
        if rects:
            painter = QPainter(pixmap)
            painter.setBrush(QColor(255, 235, 59, 130))  # สีเหลืองโปร่งแสง
            painter.setPen(Qt.PenStyle.NoPen)

            # อัตราส่วนสเกลที่ต่างกันระหว่าง PDF กับภาพ Pixmap ที่ซูม
            scale_x = pixmap.width() / page.rect.width
            scale_y = pixmap.height() / page.rect.height

            # วาดสี่เหลี่ยมไฮไลต์ลงบนจุดที่เจอคำ
            for r in rects:
                highlight_rect = QRectF(
                    r.x0 * scale_x,
                    r.y0 * scale_y,
                    (r.x1 - r.x0) * scale_x,
                    (r.y1 - r.y0) * scale_y
                )
                painter.drawRect(highlight_rect)

            painter.end()

    # แสดงภาพหน้ากระดาษลงบน Label
    self.lbl_pdf_page.setPixmap(pixmap)
```

- Highlight คำให้เป็นสีเหลืองอ่อน

ฟังก์ชันเพิ่มเติม

- **ฟังก์ชันปากกา**

สามารถขีดเขียนเส้นเป็นส่วนเสริมได้และก็สามารถไฮไลท์เพื่อช่วยในการหาคีย์เวิร์ดที่ชัดเจนและมองเห็นได้ชัดเจนขึ้น

ปัญหาและแนวทางแก้ไข

ปัญหาที่พบ

- หน้า PDF ค้างที่รูปสมุด
- Windows โหลดไดรเวอร์ไม่ได้
- คิวไฟล์ถูกล็อกบน Windows

การแก้ไขปัญหา

- ปรับให้แสดงหน้า PDF ก่อนเติมโน้ตและผล AI ลงหน้าต่าง ทดสอบเปิดเอกสารหน้าเดียวแล้วตรวจภาพในช่องกลาง
- เมื่อ DLL ที่มากับ psycopg ถูกบล็อก ตัวเปิดแอปค้นหา libpq.dll จาก PostgreSQL ที่ติดตั้งในเครื่อง
- ปิดการเชื่อมต่อ SQLite หากตั้งค่าเริ่มต้นลมเหลว ตรวจการอ่านไฟล์ส่งออกภาษาไทยด้วย UTF-8

การนำไปประยุกต์ใช้งาน

กลุ่มเป้าหมาย

- กลุ่มนักศึกษา
- งานเอกสาร

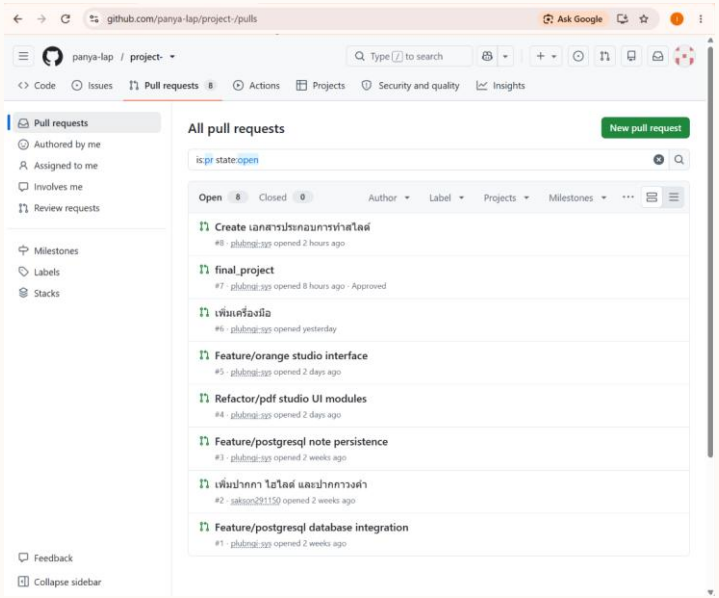
ฟีเจอร์อื่น ๆในอนาคต

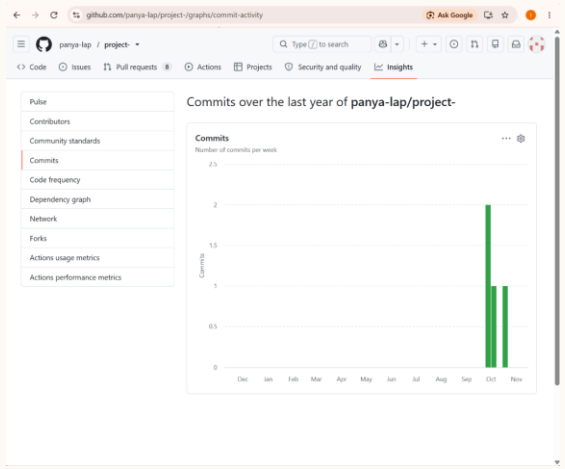
- เพิ่มระบบถาม-ตอบอัจฉริยะ ผู้ใช้สามารถพิมพ์คุยกับข้อความทั้งหมดที่เคยบันทึกไว้ใน PostgreSQL ได้ โดยมี Ollama เป็นตัวเรียบเรียงคำตอบ
- เมื่อบันทึกข้อความใหม่ ระบบจะส่งให้ Ollama วิเคราะห์และแปะแท็กจัดหมวดหมู่อัตโนมัติ (เช่น #การเรียน, #งาน) ลงฐานข้อมูลทันที
- ดึงข้อความสรุปมาให้ AI เจาะเป็นคำถาม-คำตอบ แปลงเป็นบัตรคำศัพท์ช่วยจำสำหรับนักศึกษา

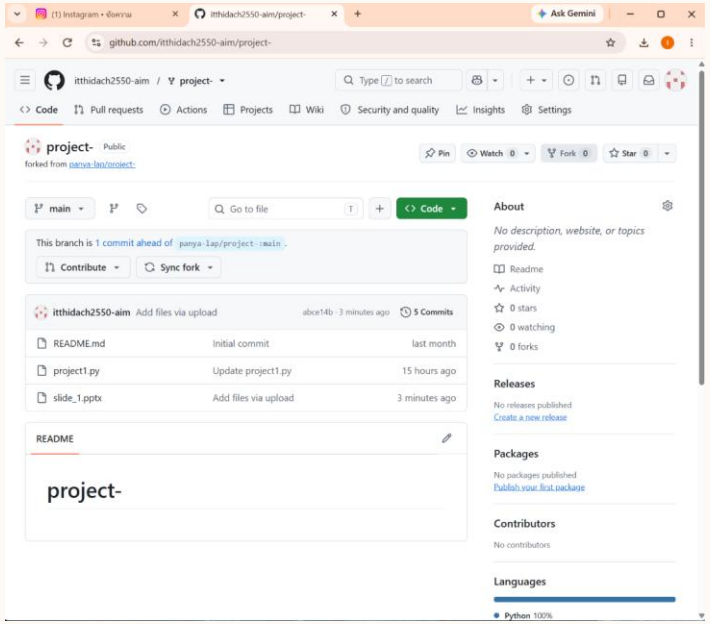
ใช้กับการเรียนได้ทั้งในห้องและก่อนสอบ

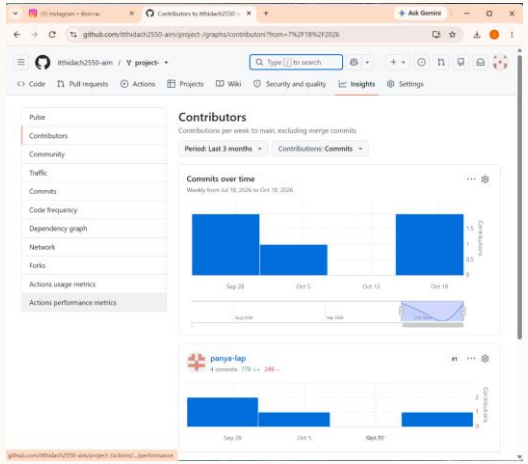
ระหว่างเรียน SQL	จดว่าเหตุใด LEFT JOIN จึงคืนแถวของแผนกที่ไม่มีพนักงาน บันทึกไว้กับหน้าตัวอย่างตาราง
ระหว่างทำแลบ	ปักธงหน้าที่มีขั้นตอนซับซ้อน ใส่คำถามที่ต้องถามอาจารย์ แล้วกลับมาปิดธงเมื่อทำซ้ำได้
ก่อนสอบ	ค้นคำที่เคยจด เช่น JOIN เลือกโน้ตที่เกี่ยวข้องหลายไฟล์ รวมเป็นชีทสั้นและทบทวนด้วยแฟลชการ์ด

การทำงานบน GITHUP









ให้แต่ละคนแนะนำตัวก่อนเล่าเรื่อง branch

panya-lap

main · เริ่ม repository และอัปเดต project1.py
ชื่อผู้เขียน 69030358_panya เชื่อมกับบัญชี panya-lap

sakson291150

sakson291150-patch-1-ปากกา · PR #2
เพิ่มปากกา ไฮไลต์ และวงคำ อยู่ใน PR แยกจาก final_project

plubngi-sys

feature/postgresql-note-persistence → final_project
เพิ่มการจัดเก็บโน้ต เครื่องมือทบทวน และชุด Windows

ประวัติการพัฒนา 2026-09-27 ถึง 2026-10-25

วันและเวลา	ผู้เขียนใน Git	Commit	งานที่เปลี่ยน
2026-09-27 19:21:58	panya-lap	42df16b	เริ่มโครงงาน
2026-09-27 19:22:58	panya-lap	34a0170	เพิ่ม project1.py
2026-10-07 19:48:27	panya-lap	5903d41	ปรับโปรแกรมใน project1.py
2026-10-23 22:37:32	69030358_panya	16d2383	แก้ project1.py
2026-10-24 15:34:23	69030229	304cede	ปากกา ไฮไลต์ และวงคำ (PR #2)
2026-10-25 11:21:14	plubngi-sys	2076e20	เพิ่ม PostgreSQL และ schema
2026-10-25 11:32:28	plubngi-sys	55b6e49	ปรับคำอธิบายโค้ดภาษาไทย

ประวัติการพัฒนา 2026-10-25 ถึง 2026-11-08

วันและเวลา	ผู้เขียนใน Git	Commit	งานที่เปลี่ยน
2026-10-25 11:32:37	plubngi-sys	be2ce88	ปรับคำอธิบายโค้ดภาษาไทย
2026-10-25 11:32:46	plubngi-sys	7f2cea6	ปรับคำอธิบายโค้ดภาษาไทย
2026-10-25 11:32:55	plubngi-sys	531d2d0	ปรับคำอธิบายโค้ดภาษาไทย
2026-10-25 11:33:04	plubngi-sys	9a25fd7	ปรับคำอธิบายโค้ดภาษาไทย
2026-10-29 22:16:33	plubngi-sys	825dbc7	บันทึกโน้ตและประวัติ AI
2026-11-08 19:05:58	plubngi-sys	b840eef	เพิ่มคิวกู้คืนและ timeout
2026-11-08 19:59:51	plubngi-sys	d91033a	แยกแท็บอ่านและจดโน้ต
2026-11-08 20:02:42	plubngi-sys	3a51df9	แยกโมดูลหน้าต่างและธีม

ประวัติการพัฒนา 2026-11-08 ถึง 2026-11-10

วันและเวลา	ผู้เขียนใน Git	Commit	งานที่เปลี่ยน
2026-11-08 20:18:49	plubngi-sys	ed784cd	ปรับพื้นที่อ่านและโทนสี
2026-11-08 22:15:37	plubngi-sys	c3ad065	ปรับธีมและเครื่องมืออ่าน
2026-11-09 00:17:03	plubngi-sys	22690bc	ค้นหาโน้ตและผล AI ข้ามไฟล์
2026-11-09 20:32:17	plubngi-sys	b7322ad	เพิ่มเครื่องมือทบทวน 4 รายการ
2026-11-09 23:19:57	plubngi-sys	15301ee	ปรับธีมกระดาษครีม
2026-11-10 13:48:46	plubngi-sys	302a65d	แก้การแสดง PDF ตรงกลาง
2026-11-10 13:56:37	plubngi-sys	01a3b05	จัดชุดแอป Windows และคู่มือ
2026-11-10 13:59:12	plubngi-sys	f1ab946	แก้ไฟล์คิวและทดสอบภาษาไทย

THANK YOU